

Internet of Things (20MCA281)

Module 2

Module 2 Syllabus

Introduction to Fog Computing: principles, architectures, and applications. TinyOS – NesC, Programming frameworks for Internet of Things

Introduction

- IoT environments consist of loosely connected devices that are connected through heterogeneous networks
- The purpose is collecting and processing data from IoT devices to mine and detect patterns, or perform predictive analysis or optimization and finally make smarter decision in a timely manner

- Data in such environment can be classified to two categories
 - Little Data or Big Stream: transient data that is captured constantly from IoT smart devices
 - Big data: persistent data and knowledge stored and archived in centralized cloud storage
- IoT environments including smart cities and infrastructures need both Big Stream and Big Data for effective real-time analytics and decision making

- At the moment, data is collected and aggregated from IoT networks that consist of smart devices and is sent uplink to cloud servers where it is stored and processed.
- Cloud computing offers a solution at the infrastructure level that supports Big Data Processing
- It enables highly scalable computing platforms that can be configured on demand to meet constant changes of applications requirements in a pay-per-use mode, reducing the investment necessary to build the desired analytics application.

- This perfectly matches requirement of Big Data processing when data is stored in centralized cloud storage.
- In such case, processing of large magnitude of data volume is enabled by on-demand scalability of Clouds.
- However, when data sources are distributed across multiple locations and low latency is absolutely necessary, in-cloud data processing fails to meet the requirements

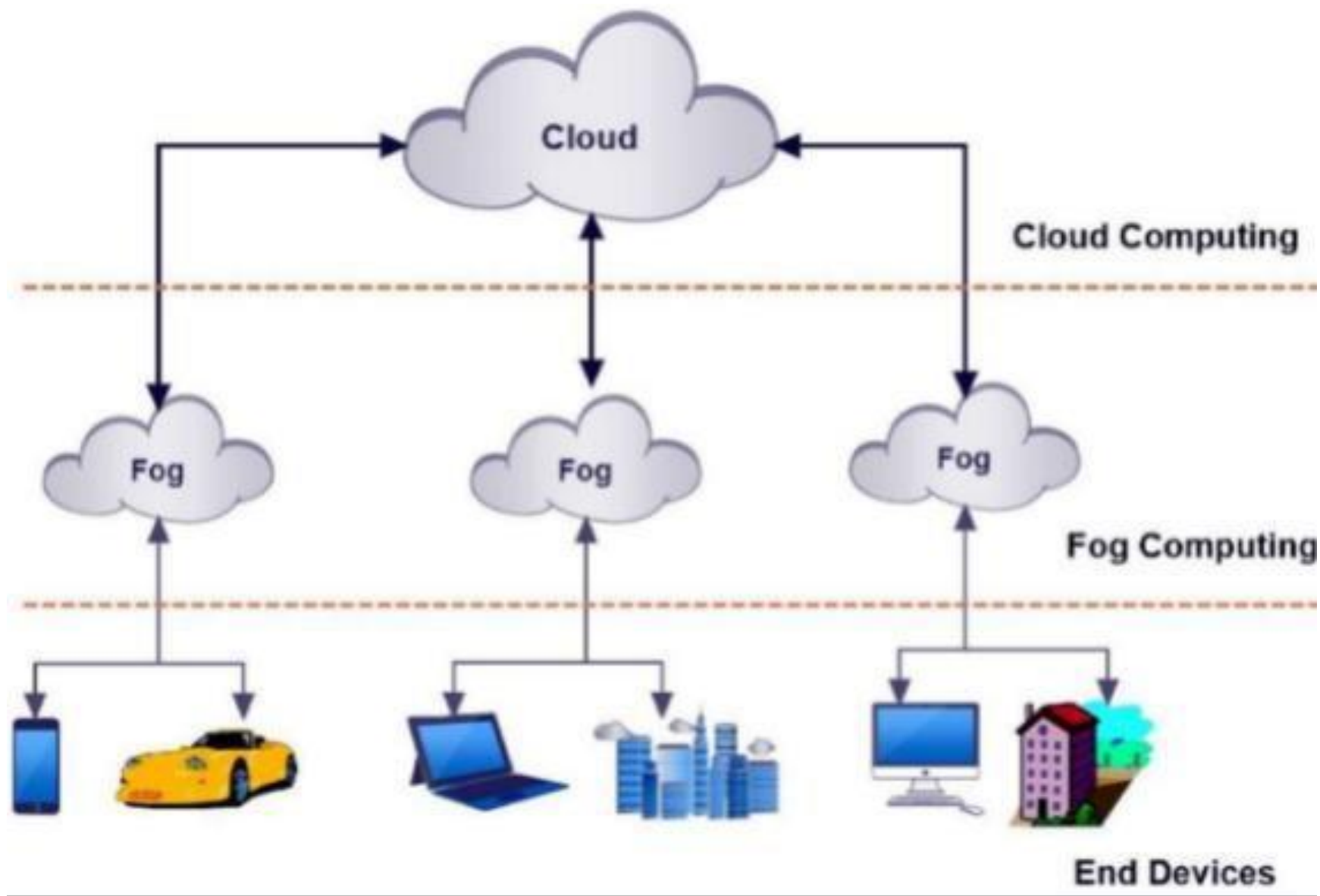
Motivation

- A recent analysis of Endomondo application, a popular sport activity tracking application has revealed number of remarkable observations.
- The study shows that a single workout generates 170 GPS tuples
- Total number of GPS tuples can reach 6.3 million in a month time

- With 30 million users data flows of Endomondo can reach up to 25,000 tuple per second
- GPS sensors in cars to air and noise pollution sensors—can easily reach millions of tuples per second
- Centralized cloud servers cannot deal with flows with such velocity in real-time
- Considerable numbers of users are not comfortable to transfer and store activity track data into the cloud due to privacy concern

- This motivates the need of alternative paradigm that is capable of bringing the computation to more computationally capable devices that are geographically closer to the sensors
- Such devices, which are in the *edge of the network* and therefore referred to as *edge devices*

- Edge devices can build local views of data flows and can aggregate data to be sent to the cloud for further offline analysis.
- To this end, Fog computing has emerged



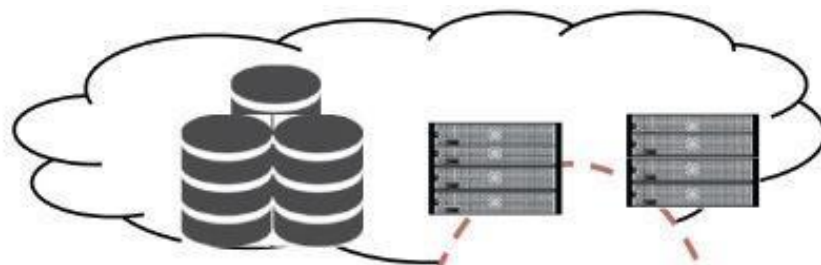
Definitions and Characteristics

- Fog computing is defined as a distributed computing paradigm that fundamentally extends the services provided by the cloud to the edge of the network
- It facilitates management and programming of compute, networking and storage services between data centers and end devices

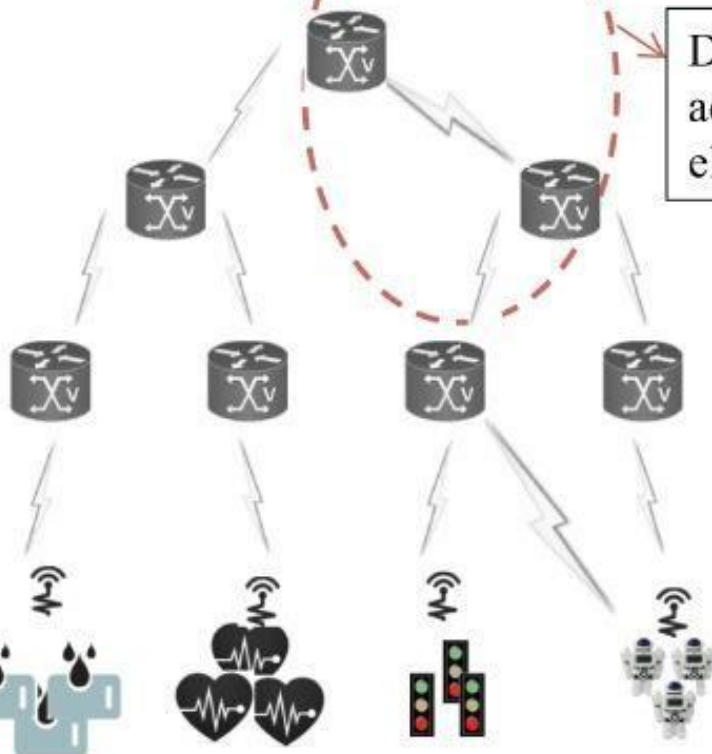
- Involves components of an application running both in the cloud as well as in edge devices between sensors and the cloud, i.e. in smart gateways, routers or dedicated fog devices
- Fog computing supports:
 - Mobility
 - Computing resources
 - Communication protocols
 - Interface heterogeneity
 - Cloud integration
 - Distributed data analytics

to addresses requirements of applications that need low latency with a wide and dense geographical distribution.

Cloud Computing
Resources



Edge Devices and
Network Elements



Distributing computing
across cloud and network
elements dynamically.

Sensors

How Data Flows



1. Generate Data

A smart device creates time-sensitive data.

2. Local Analysis

A nearby "Fog Node" processes the data instantly.

3. Act Locally

A quick, real-time action is taken.



4. Summarize

Only essential summaries are sent to the Cloud.

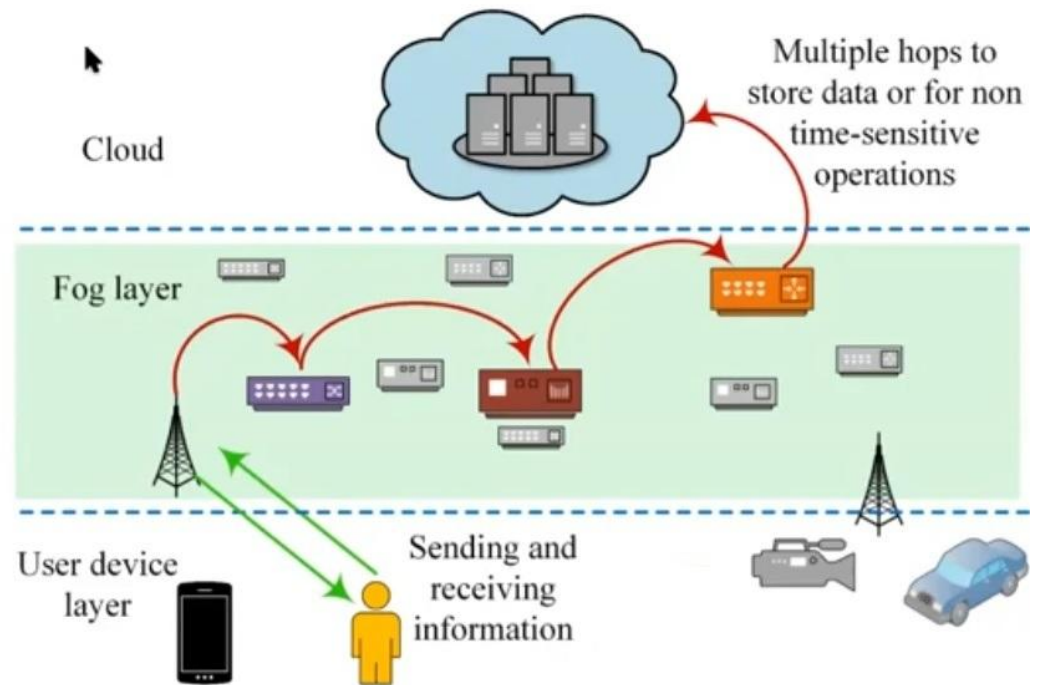


Advantages of FOG computing

1. Low latency and drastic reduction in amount of data that is sent to the cloud.
2. Since the distance to be travelled by the data is reduced, it results in saving network bandwidth.
3. Improves scope for real time application and reduces the response time of the system. This results in enhancing user experience
4. System security is improved in addition to better privacy since the data resides close to the host.

Advantages

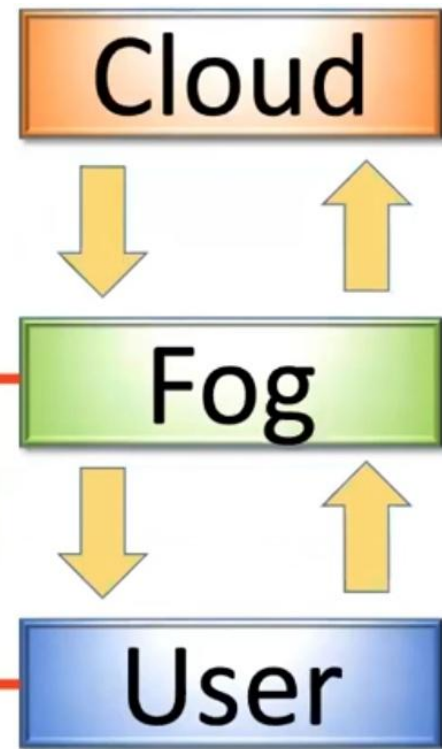
- Minimize latency
- Bandwidth conservation
- Enhanced security
- Reliable operations
- Spatially aware data
- Optimized movement of data



Minimize Latency

- Time-sensitive operations
- Operation on sensor data closer to end devices
- Reduced transmission time
- Tradeoff between execution speed of cloud and fog

Execution of
time-sensitive
data



*“Analyzing data close to the device that collected the data can make the difference between **averting disaster** and a **cascading system failure**.”*

-Cisco

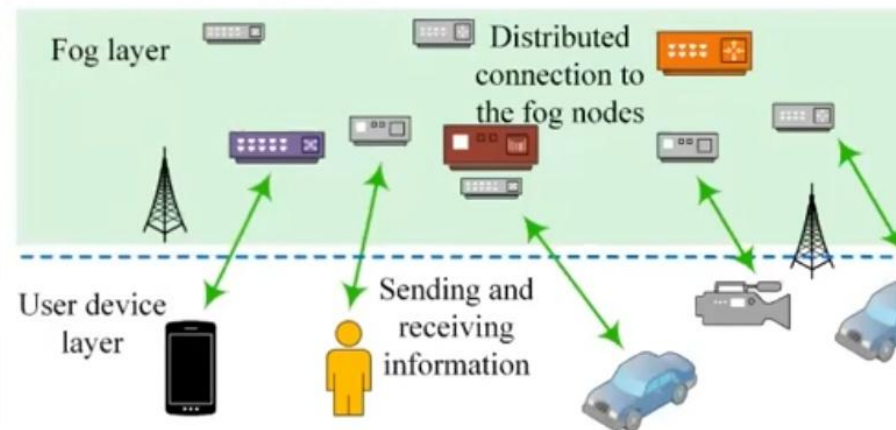
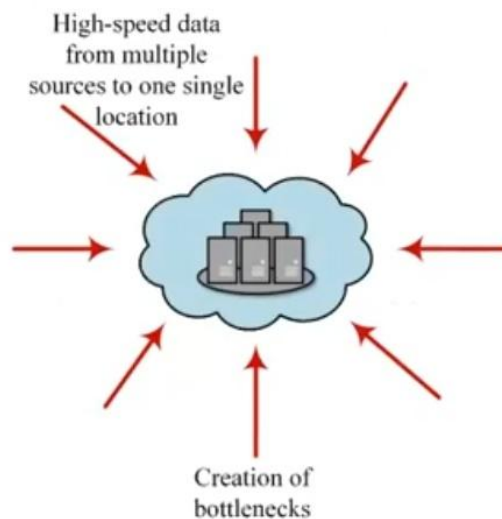
Low latency requirement:

- Mission critical applications require **real-time data processing**.
- Examples of such applications is cloud robotics, or **anti-lock brakes on a vehicle**
- For a robot, motion control depends on the data collected by the sensors and the feedback of the control system

- Having the control system running on the cloud may make the **sense-process-actuate loop slow** or unavailable as result of communication failures.
- This is **where fog computing helps by performing the processing required for control system very close to the robots** - thus **making real-time response possible**

Bandwidth Conservation

- IoT devices generate huge amount of data
- Not practical to send everything to the cloud
- Not all applications need cloud level processing/storage
- Possibility of bottlenecks
- Reject/postpone services



Enhanced Security

- Data travels through multiple networks to reach the cloud
- Spatially, data travels long distances
- Fog allows local processing
- Data does not need to travel far
- Remains closer to data generating sensor modules
- Reduces the possibility of attacks
- Limited to local network

Reduction of network traffic:

- More than 50 billion connected devices are there
- Billions of mobile devices such as smart phones and tablets already being used to generate, receive and send data make a case for putting the **computing capabilities** closer to where devices are located rather than having all data sent over networks to central data centers

- Therefore, it is neither efficient nor sensible to send all of this raw data to the cloud
- Hence, fog computing benefits here by providing a platform for filter and analysis of the data generated by these devices close to the edge, and for generation of local data views
- This drastically reduces the traffic being sent to the cloud

Suitable for IoT tasks and queries:

With the increasing number of smart devices, most of the requests pertain to the surroundings of the device

Such requests can be served without the help of the global information present at the cloud

Eg., Endomondo application allows a user to locate people playing a similar sport nearby

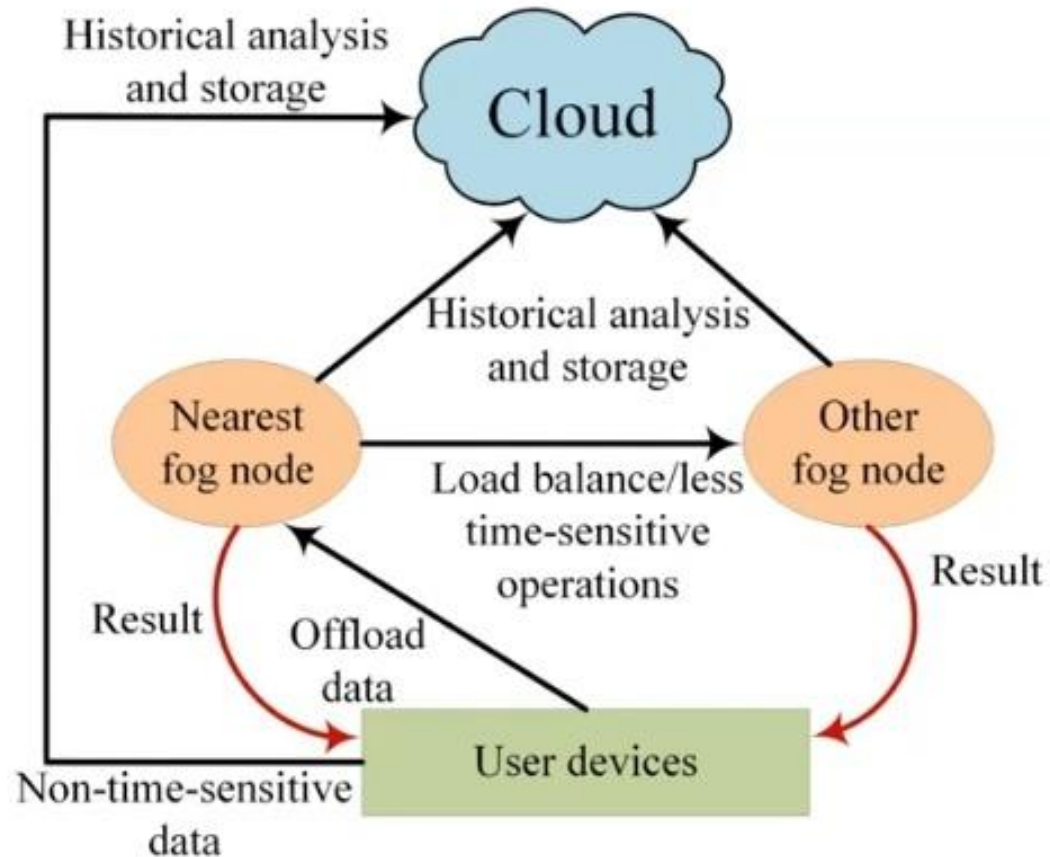
- Because of the **local nature of the typical requests** made by this application, it makes sense that **the requests are processed in fog rather than cloud infrastructure**.
- Another example can be a **smart connected vehicle which needs to capture events only about a hundred meters from it**.
- Fog computing makes the communication distance closer to the physical distance by bringing the processing closer to the edge of network

Scalability:

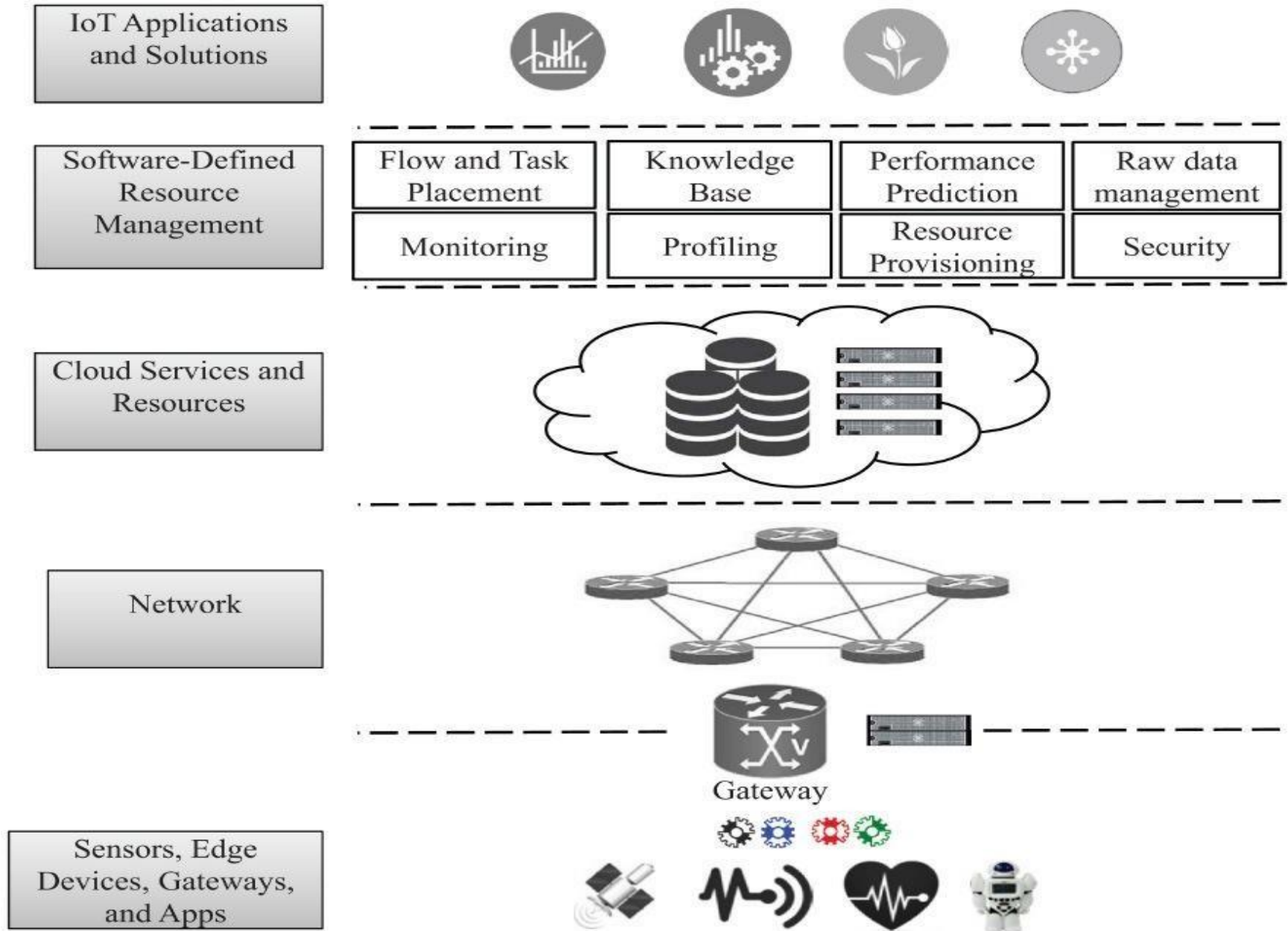
- Even with virtually infinite resources, the cloud may become the bottleneck if all the raw data generated by end devices is continued to be sent to it
- Fog computing aims at processing incoming data closer to the data source itself, it reduces the burden of processing that on the cloud, thus addressing the scalability issues arising out of the increasing number of endpoints

Other Advantages...

- Reliable operations
- Spatially aware data
- Optimized movement of data
- Reduce load from the cloud
- Support mobility



Fog - Reference Architecture



- 1. IOT DEVICE (Edge) Layer

In the **bottommost layer** lays the **end devices (sensors)**, as well as **edge devices and gateways**

- This layer **also includes apps** that can be installed in the end devices to enhance their functionality

- 2. Network Layer

Elements from this layer use the next layer, the network, for communicating between themselves, and between them and the cloud

- 3. Cloud services Layer

The next layer contains the cloud services and resources that support resource management and processing of IoT tasks that reach the cloud.

- 4. Software defined resource management layer

On top of the cloud layer lays the resource management software that manage the whole infrastructure and enable quality of Service to Fog Computing applications

- The topmost layer contains the applications that leverage fog computing to deliver innovative and intelligent applications to end

- The Software-Defined Resource Management layer implements many middleware-like services to optimize the use of the cloud and Fog resources on behalf of the applications
- The goal of these services is to reduce the cost of using the cloud at the same time that performance of applications reach acceptable levels of latency by pushing task execution to Fog nodes
- This achieved with a number of services working together, as follows

- ***Flow and task placement:***
- This component keeps track of the state of available cloud, Fog and network resources to identify the best candidates to hold incoming tasks and flows for execution
- Communicates with the Resource Provisioning service to indicate the current number of flows and tasks, which may trigger new rounds of allocations if deemed too high

- ***Knowledge Base:***

- This component stores historical information about application demand and resource demands that can be leveraged by other services to support their decision-making process

- ***Performance Prediction:***

- This service utilizes information of the Knowledge Base service to estimate the performance of available cloud resources

- This information is used by the Resource Provisioning service to decide the amount of resources to be provisioned
- *Raw Data Management:*
- This service has direct access to the data sources and provides views from the data for other services
- These views can be obtained by simple querying (Eg., SQL, NOSQL etc.,) whereas other times more complex processing may be required (e.g, MapReduce)

- *Monitoring.*

- This service keeps track of the performance and status of applications and services and supplies this information to other services as required

- *Profiling.*

- This service builds resource and applications profiles based on information obtained from the Knowledge Base and Monitoring services

- ***Resource Provisioning:***

- This service is responsible for acquiring cloud, Fog and network resources for hosting the applications
- This allocation is dynamic, as requirements of applications, as well as number of hosted applications, changes over time
- Decision on the number of resources is made with use of information provided by other services and user requirements on latency as well as credentials managed by the Security service

- Example, the component pushes tasks with low latency requirements to edge of network as soon as free resources are available
- ***Security:***
- This service supplies authentication, authorization, and cryptography as required by services and applications

Disadvantages

- Deployment complexity
- Complex architecture
- Higher maintenance
- Security challenges
- Cost
- Resource management difficulty

Applications

- **Healthcare:**
- A fog computing assisted distributed analytics system to monitor fall for **stroke patients**
- Designed a real-time fall detection system based on fog computing that divides the fall detection task between edge devices and the cloud

- Another proposal is a three-tier architecture for smart-healthcare infrastructure.
- Comprising of a role model, layered cloud architecture, and a fog-computing layer in order to provide an efficient architecture for healthcare and elderly-care applications
- The fog layer improves the architecture by providing low latency, mobility support, location awareness, and security measures

- **Augmented Reality:**
- **Highly latency-intolerant** as even very small delays in response can damage the user experience
- Hence, **fog computing has the potential to become a major player in the augmented reality domain**
- Brain state classification (the most computationally heavy signal processing tasks) can be done using this

- Another example is **Wearable computer system based on Google Glass** devices that assists people with reduced mental ability
- The compute-intensive workloads of this application need to be offloaded to an external server
- This offloading must provide crisp, real-time responses

- **Caching and Preprocessing:**

- Edge servers can be used for improving web sites performance
- Users connect to the internet through fog boxes - hence each HTTP request made by a user goes through a fog device
- A number of optimizations are done that reduces the amount of time the user has to wait for the requested web page to load
- Edge devices also perform optimizations that take user behavior and network conditions into account

- In case of **network congestion**, the **edge device may provide low resolution graphics** to the user to reach acceptable response times
- The edge device can also monitor the performance of the client machines
- Important use is the integration of IoT and Cloud computing
- This **trimming or pre-processing** of data before sending it to the cloud will be a **necessity** in IoT **environments because of the huge amount of data** generated by these environments

- Sending huge volumes of raw data to the cloud will lead to both core network and data center congestion
- A smart gateway based communication for integrating IoT with cloud computing is proposed to meet the challenge
- Data generated by IoT devices is sent to the smart gateway
- The smart gateway handles the pre-processing required before sending the data to the cloud
- The smart gateway is assisted by fog computing services

Tiny Operating System – (TinyOS)

- An *operating system* (OS) can be defined as the master program existing between the computer hardware and the user
- Its main purpose is the management and coordination of various system resources
- Resources:
 - H/W – Hard disk, Memory, Peripheral devices etc.,
 - Non H/W – managing processor time slot

- It provides equal access to these resources through numerous system interfaces
- An operating system can act as a host on which other programs can be executed
- Provide systemwide services by which these programs can access shared resources
- OS services includes
 - Networking services
 - File I/O
 - Data security
 - Provision of user interfaces

Types of OS

- Real-time
- Embedded
- Single-user and
- Multiuser operating systems

- In most wireless sensor networks, sensors implement a basic operating system containing the minimum functions necessary to execute their tasks
- This minimalist approach is of great importance because of the highly resource-constrained nature of these sensing devices
- Operating system code and application code must reside in devices having much less than one megabyte (1 MB) of memory

- Sensor-based operating systems typically provide two basic services:
 1. A hardware abstraction layer for accessing the various sensing devices
 2. A network interface for communication
- There are several sensor-based OSes in use today; some of these are TinyOS, Contiki, Mantis OS, BTnut, and Nano-RK

- **TinyOS**, as its name implies, can be described as a miniature framework **designed for embedded systems that require very aggressive resource management**
- This is due to the **highly constrained nature of their resources such as power and available memory**
- It implements the hardware abstraction layer and scheduler of a conventional operating system

- Allows generic programs that may have no knowledge of the details of the operations supported by the underlying hardware components (such as sensors) to use well-defined interfaces to interact with these components
- Its C-language-like framework provides an interface to core system components
- Allows a programmer to manage various services of the system

Components of TinyOS

- TinyOS provides software abstraction for hardware components such as its communication, routing, sensing, and storage subsystems.
- *Software components* in TinyOS refer to abstractions of specific services either provided by either another software component or a hardware component

- A software component consists of any number of the following:
 - Modules
 - Configurations
- **Modules:**
- Lowest form of abstraction provided by the TinyOS operating system
- They implement program logic that directly addresses a software component and can also provide a particular set of services, thereby enabling the reuse of software components

- Other software components can interact with a module through its defined interface, which specifies a set of operations/services that a particular module implementation provides
- **Configurations:**
- **Abstractions of multiple modules** and other configurations grouped together form a newly abstracted component referred to as a *configuration*

- A configuration can be **visualized as a super component** consisting of several subcomponents to **provide a single unified interface**
- Configurations wire a set of components de-fined in a component signature
- This is the set of interfaces that a component uses and provides to another component
- Thereby allowing two or more components to communicate with each other

Introduction to NesC

- In TinyOS the software component abstractions are written in a dialect of the C language called *nesC*.
- NesC contains a subset of features of the standard C libraries and syntax with some extensions such as commands, and events added to accommodate its event-driven approach to programming rather than the traditional imperative style of many C-based languages

- Due to the resource constrained environments typical of most sensors, these additional features in nesC help improve the efficiency of programs written in the language

FRAMEWORK^{TinyOS}

Communication
Subsystem

Sensing
Subsystem

Storage
Subsystem

Power
Subsystem

Tire Interface



Tire Module

Engine Interface

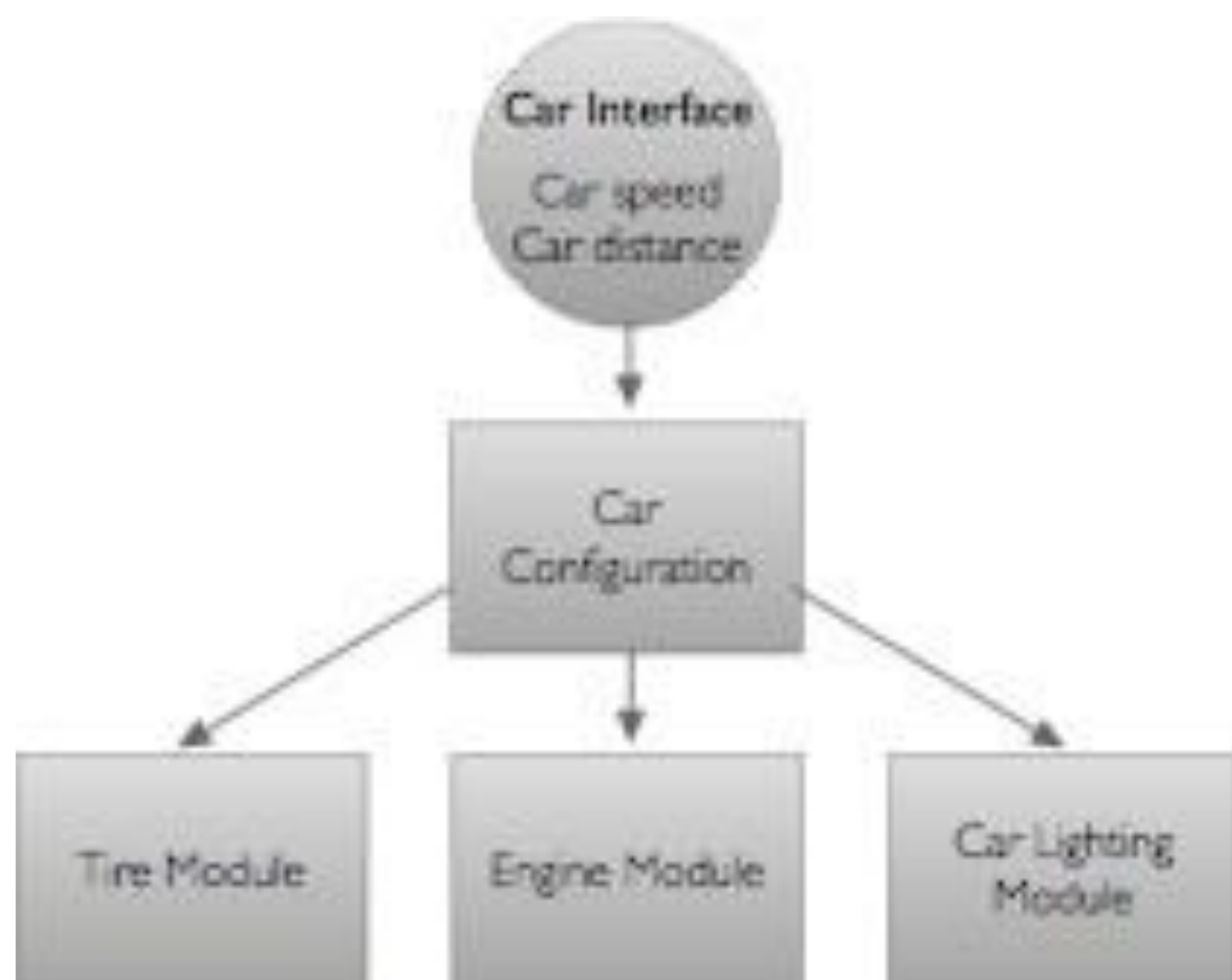


Engine Module

Light Interface



Car Lighting
Module



Split-Phase Operations: Commands and Events

- Functions can be declared as commands, events or tasks

command <**return** type> <command name>
(<input parameters>) commanderrort
readSensor()

event **void** <event name> (<returned messages
and errors>) event **void** readDone (error t result,
uint16 t value)

- Commands are functions already implemented by the called software component in which its invocation and completion paths have been disassociated
- Most commands when called return immediately without completion, and if any errors or messages result at the end of its computation, a callback function is signaled
- Having a separate callback function signaled after the completion of a command is called *split-phase*

- TinyOS contains no blocking operations; therefore any long-running operation can immediately return control to the calling program, and on completion a callback function is signaled
- ***A command and Its Associated Callback Function :***
- Events are triggered in response to some action; this may be after the execution of a command or in response to messages from the environment

- Eg., the reception of a packet, triggers an event *receive*
- Events are not implemented by the called component but rather are functions whose implementation details are left to the calling software component
- Events are methods that must be overridden by the caller
- All callback functions signaled after the termination of a command are events

- ***Code Sample Illustrating the Split-Phase Concept :***
- In the blocking portion of the code, the program does not increment the ***SendCount*** variable until the Read operation has completed and the conditional statement evaluated
- semantics are very different in the split-phase version of the code, in which the ***Read*** command is called but does not halt the program flow

- When the reading operation is finalized, the event ***ReadDone*** is signaled with the value of the reading and a message indicating whether the reading is a success or failure

Tasks

- Represents the TinyOS notion of concurrency
- It allows a function call to be deferred for a later time when the scheduler can process less important tasks
- Multiple tasks can be queued up in the task queue, which is a FIFO (First-in/First-out) data structure.
- Unlike commands and events, a running task can be preempted by the system when an interrupt occurs

```
// Blocking Non Split-Phase if (Read () ==  
SUCCESS)  
{  
    SendCount++;  
}  
  
//NonBlocking Split-Phase //Start Phase  
Sensor.Read ();
```

//Complete phase

event **void** ReadDone(error t err, uint16 value)

{

if (err == SUCCESS)

 {

 printf("Value read by sensor is \\\%d",
value);

 }

}

Syntax of task function

task <**return** type> <task name> (<input param>)

task **void** computation (**void**)

EVENT-DRIVEN PROGRAMMING

- Refers to a programming paradigm in which program flow is determined by events
- In wireless sensor networks, where conservation of computational and energy resources is paramount, event-based programming is very critical
- Programs running on these sensing devices would have to continuously poll the system checking for the occurrence of certain events

- This approach is both wasteful and ineffective in larger sensor network
- Causes program degradation
- Event-driven programming approach allows for fine-grained control over power usage while maintaining the scheduling flexibility required in sensor networks due to their unpredictable nature

Event Triggering and Handling in nesC

```
if (call AMSend. send (packet, sizeof (Data))) {  
    busy = TRUE;  
}  
  
event message t* Receive.receive (message t* msg,  
void* pay- load, uint8_t len)  
{  
    call Leds.led0Toggle;  
    busy = FALSE  
  
}
```

Programming Frameworks for IoT

- IoT devices are characterized as small things in the real world with limited storage and processing capacity
- May not be capable of processing a complete computing by themselves
- May need computational capabilities of a cloud-based back-end to process the task and web-based front end to interact with users

- Cloud infrastructure supports for:
 - Device virtualization
 - Availability
 - Provisioning of resources
 - Data storage
 - Data analytics
- IoT creates a new avenue for computing with:
 - Huge storage capacity
 - Massive computing capabilities (collect, analyze, process and archive the data)
 - Provides SaaS (Sensing as a Service), SAaaS (Sensing and Actuation as a Service) and VSaaS (Video Surveillance as a Service)

IoT Challenges

- Heterogeneity and the volume of data generated are two of the biggest concerns
- Heterogeneity spans through hardware, software, and communication platforms.
- The data generated from these devices are generally in huge volume, are in various forms, and are generated at varying speeds

- Some of the IoT applications such as traffic management will be latency-sensitive, and this warrants edge-processing support by the programming framework
- Another difficulty is how to cope with frequent periods of non-availability of devices, caused by mobilities and limited energy supplies from batteries
- A simplified programming model is needed that can provide solutions for the above set of challenges

Overview

- During the lifecycle of IoT applications, the footprint of an application and the **cost of its language** runtime play a huge role in the sustainability of an application.
- **C** has been used predominantly in embedded applications development due to its **performance**
- It can occupy the same position in IoT programming too

- The choice of **communication protocols** also has a huge implication in the cost of IoT applications on devices
- **Remote Procedure Calls (RPC), Representational state transfer (REST), and Constrained Application Protocol (CoAP)** are some of the communication methods that are being currently incorporated into IoT communication stacks
- **Need a coordination language that can manage communications between various components of an IoT ecosystem**

- An explicit coordination language can tackle many of the challenges
- It can manage:
 - Communication between heterogeneous devices
 - Coordinate interaction with the Cloud and devices
 - Handle asynchronous data arrival
 - Provide support for fault tolerance
- The method of using more than one language in a given application is known as polyglot programming.
- Polyglot programming is being widely used in web applications development, provides the same advantages for IoT programming too

Embedded Device Programming Languages

- More than 80% projects, are either implemented in C and its flavors, or in a combination of C and other languages such as C++
- C features:
 - Performance
 - Small memory foot-print
 - Access to low-level hardware
 - Availability of a large number of trained/experienced C programmers
 - Short learning curve
 - Compiler support

- Many embedded C-compilers based on ANSI C usually:
 1. Support low-level coding to exploit the underlying hardware
 2. Support in-line assembly code
 3. Flag dynamic memory-allocation and recursion
 4. Provide exclusive access to I/O registers
 5. Support accessing registers through memory pointers, and
 6. Allow bit-level access.
- nesC, Keil C, Dynamic C, and B# are some of the flavors of C used in embedded programming

Message Passing in Devices

- **Remote Procedure Call(RPC):**
- Is an abstraction for procedural calls across languages, platforms, and protection mechanisms
- RPC can support communication between devices as it implements the request/response communication pattern

- When RPC messages are transported over the network, all the parameters are serialized into a sequence of bytes
- Lightweight Remote Procedure Call (LRPC) was designed for optimized communication between protection domains in the same machine
- S-RPC is another light- weight remote procedure-call for heterogeneous WSN networks
- A lightweight RPC has been incorporated into the TinyOS, nesC environment

- **REST:**
- The main aim of the REST was to simplify the web-application development and interaction
- Stipulates the following constraints on application development:
 - Should be based on client-server architecture and the servers should be stateless
 - Support should be provided for caching at the client side
 - The interface to servers should be generic and standardized (URI)

- Layering in the application architecture should be supported and each of the layers shall be independent
- Optional code-on demand should be extended to clients having the capability
- These constraints, combined with the following principles, define the RESTful approach to application development:
 - Everything on the Internet is a resource
 - Unique identifiers are available to identify the resources

- Generic and simple interfaces are available to work with those resources
- Communication between client and servers can be through representation of resources
- Resource representation through sequence of bytes is followed by some metadata explaining the organization of the data
- Since transactions are stateless, all interactions should be context-free
- Layering is supported, and hence intermediaries should be transparent

- Essential characteristic features of a composing language that can compose RESTful components together:
 - Support for dynamic and late binding
 - Uniform interface support for composed resource manipulation
 - Support for dynamic typing
 - Support for content-type negotiation
 - Support for state inspection of compositions by the client

- RESTful approaches is limited to data and its representation
- In the Internet, the exchanges need not be limited to data and its representation
- RESTful approach poses a challenge for those applications that require stateful interactions
- CREST (Computational REST) tries to address these problems
- The focus is on exchanges of computation rather than on data exchange

- Instead of client-server nomenclature, every- one is addressed as peers; some may be strong and some may be weak, based upon the available computing power
- Functional languages such as Scheme allow computations to be suspended at a point and encapsulated as a single entity to be resumed at a later point in time, through continuation.
- CREST's focus is on these sorts of computation
- It supports the model of “computations stopping at a point in a node, exchanged with another node, resumed from the suspended point at the new node”

- CREST Principles:
 - All computations are resources and are uniquely identified
 - Representation of resources through expressions and metadata
 - All computations are context-free
 - Support for layering and transparent intermediaries
 - All the computations should be included inside HTTP
 - Computations can produce different results at different times
 - Between calls they can maintain states that may aid computations such as aggregation
 - Between different calls, computations should support independence
 - Parallel synchronous invocations should not corrupt data

- **CoAP:**
- HTTP/TCP stack is known to be resource demanding on constrained devices
- Protocols such as Embedded Binary HTTP (EBHTTP) and Compressed HTTP Over PAN (CHoPAN) have been proposed
- The issue of reliable communications still remains a concern
- CoAP is optimized for constrained power and processing capabilities of IoT

- CoAP is a two-layered protocol: **a messages layer**, interacting with the UDP, and **another layer for request/response interactions** using methods and response code, as done in HTTP
- **CoAP exchanges messages asynchronously** and uses UDP.
- The CoAP has four types of messages: *Acknowledgement*, *Reset*, *Confirmable (CON)*, and *Non-Confirmable (NON)*
- Reliability is provided by the message layer and will be activated when Confirmable messages are used. The Request methods are: GET, POST, PUT, and DELETE of HTTP

Advantages of CoAP

- A compact binary header (10–20 bytes), along with UDP, reduces the communication overhead, thereby reducing the delay and minimizing the power consumption
- Since asynchronous data push is supported, it enables things to send information only when there is a change of observed state. This allows the things to sleep most of the time and conserve power.
- Less complex when compared to HTTP.
- This lowers the hardware requirements for the smart- things on which it executes.

Coordination Languages – Assignment No.1

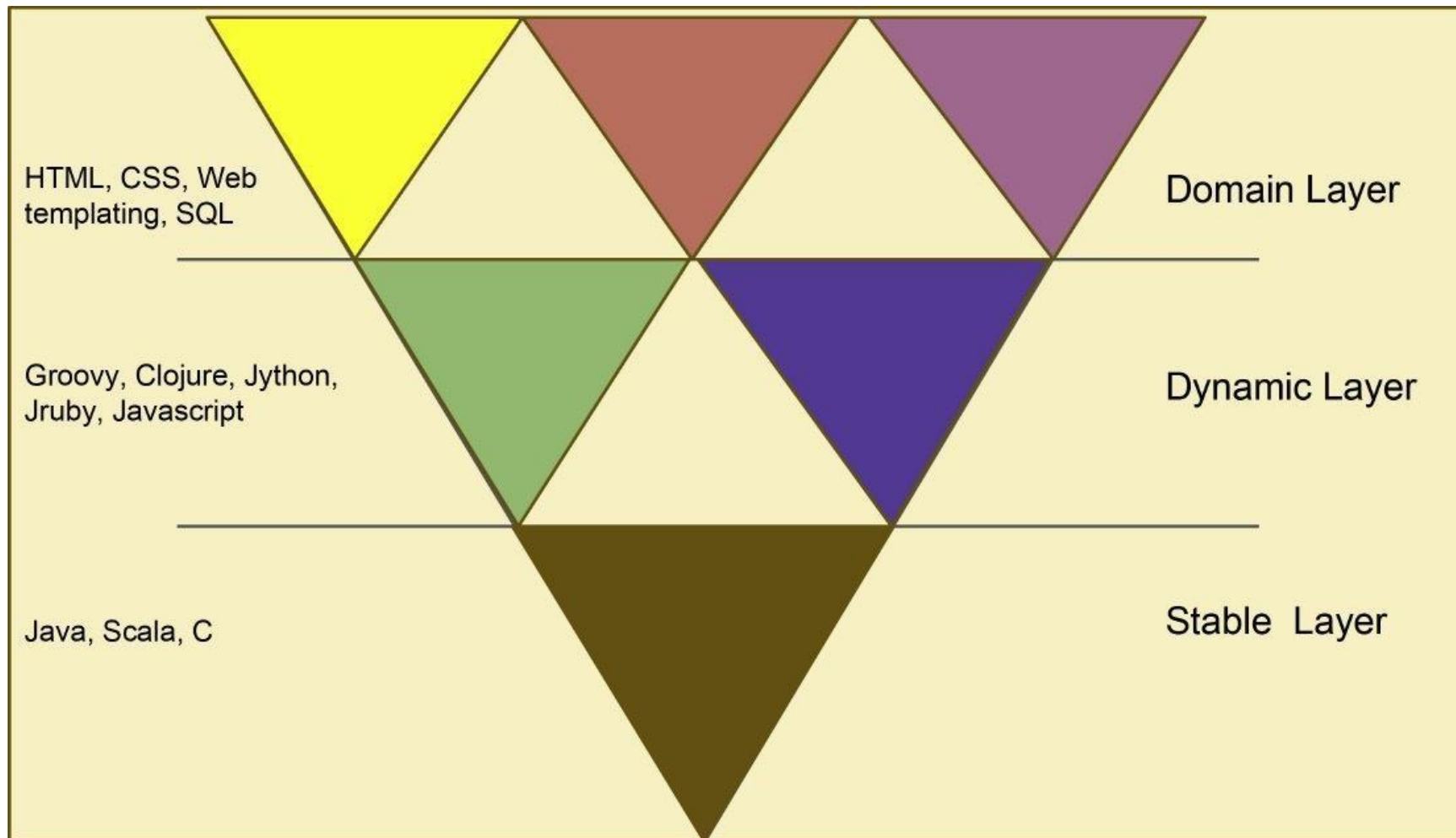
- Linda and eLinda
- Orc
- Jolie

Polyglot Programming

- Also called multilingual programming.
- It is an art of developing simpler solutions by combining the best possible solutions using different programming languages and paradigms
- This is based on the observation that there is no single programming paradigm or a programming language which can suit all the facets of modern-day programming or software requirements.
-

- Many modern-day software combines a subset of imperative, functional, logical, object-oriented, concurrent, parallel, and reactive programming paradigms
- One of the oldest examples of polyglot programming is Emacs, which is a combination of parts written in both C and eLisp
- Web applications are generally based on three-tier architecture is a representation of polyglot software systems

- The platform used for the integration, and the different programming languages supported by the given platform are the two essential aspects
- An Inverse pyramid can be used to categorize the programming languages in a polyglot software system
- The Inverse pyramid has three layers: stable, dynamic, and domain



- Statically typed programming-languages such as **Java and C** that provide well-tested and stable functionality settle toward **the stable layer**
- The **less powerful general purpose technologies**, such as HTML and CSS, which are tightly coupled to a specific part of the application, bubble up to the **top layer**
- The **dynamic layer** in the middle consists of a variety of programming languages such as Groovy and Clojure, which are more flexible and aid **rapid functionality development**

- Heterogeneous nature of IoT makes a single programming language may not be able to provide complete support for the application development
- At least a coordination language and a computational language is required in a unified programming model for IoT, which, in a way, is polyglot programming

Survey of IoT Programming Frameworks – Assignment No.2

- Overview
- IoT Programming Approaches
- Existing IoT Framework
 - *Mobile Fog*
 - *ELIoT (Erlang Language for IoT)*
 - *Compose API*
 - *Distributed Dataflow Support for IoT*
 - *PyoT*

- *Dripcast*
- *Calvin*
- *Simurgh*
- *High-Level Application Development for the Internet of Things*
- *PatRICIA*